
# The Spring Framework: Introduction to Lightweight J2EE™ Architecture

**Rod Johnson / Juergen Hoeller**

CEO / CTO

Interface21

www.springframework.org

Session TS-7695

# Goals of This Talk

Understand the "lightweight container" architecture—and how to realise it with the Spring Framework, the leading open source lightweight container

# Agenda

- **"Agile" J2EE™ technology**
  - Why we can't settle for "business as usual"
- The lightweight container movement
- Enabling technologies
  - Dependency Injection
  - Aspect Oriented Programming (AOP)
- The Spring Framework

# Agile J2EE: Where Do We Want to Go?

- Need to be able to produce high quality applications, faster and at lower cost
- Need to be able to cope with changing requirements
  - Waterfall is no longer an option
- Need to simplify the programming model
  - **Need to reduce complexity rather than rely on tools to hide it**
  - …but must keep the power of the J2EE platform

# Agile J2EE: Why Is This Important?

- Java™ technology/J2EE is facing challenges at the low end
  - .NET
  - PHP
  - Ruby
- Concerns from high end clients (banking in particular) that J2EE development is slow and expensive
- Complacency is dangerous

# Agile J2EE: Aren't We There Yet?

## Problems with traditional J2EE architecture…

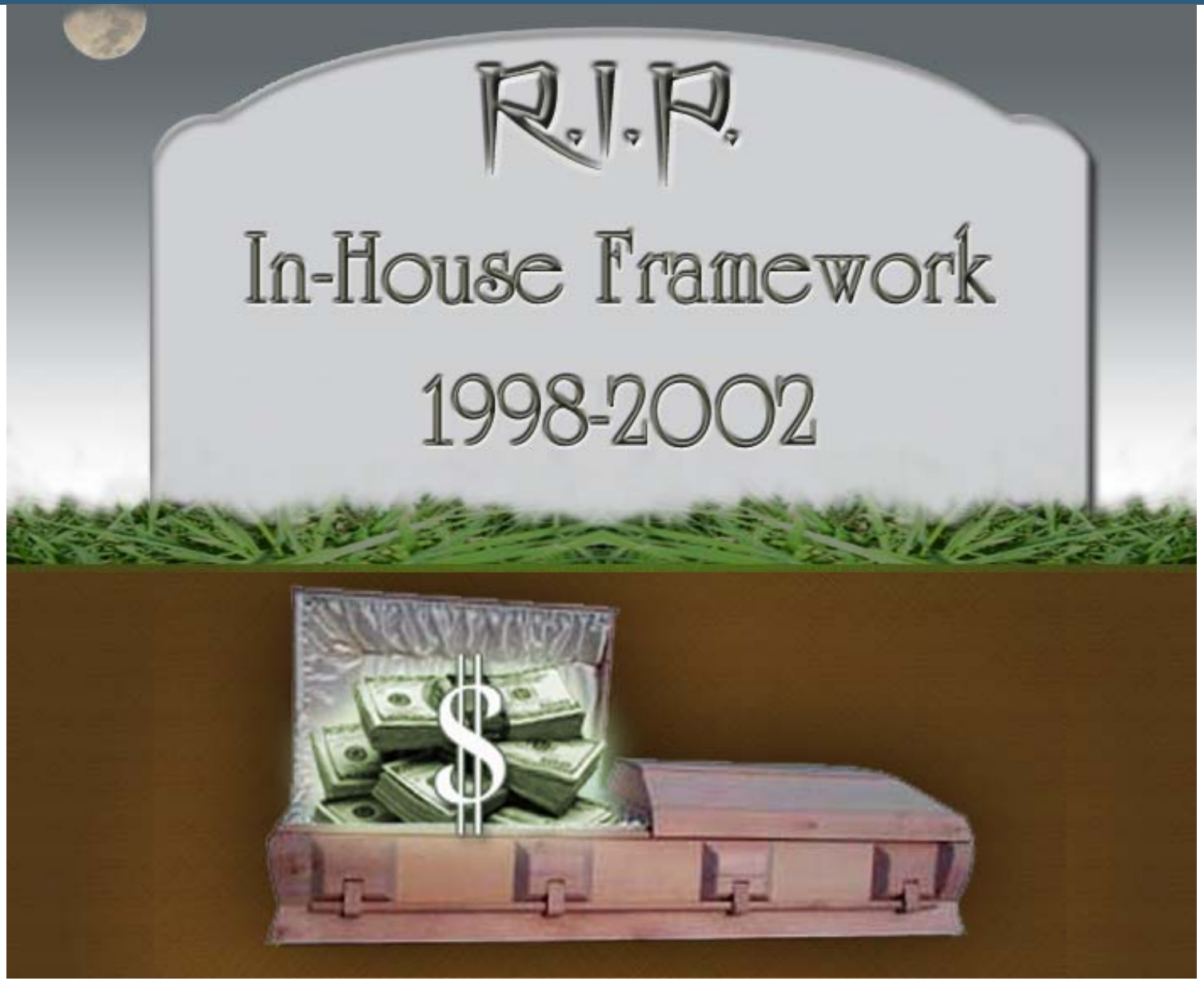
- Difficult to test traditionally architected J2EE apps
  - EJBs depend heavily on the runtime framework, making them relatively hard to author and test
- Simply too much code
  - Pet Store as an example
  - Much of that code is mundane "glue" code
- Heavyweight runtime environment
  - Components need to be explicitly deployed to be able to run, even for testing
  - Slow change-deploy-test cycle

# Agenda

- "Agile" J2EE technology
  - Why we can't settle for "business as usual"
- **The lightweight container movement**
- Enabling technologies
  - Dependency Injection
  - Aspect Oriented Programming (AOP)
- The Spring Framework

# Enter Lightweight Containers

- **Frameworks** are central to modern J2EE development
- Many projects encounter the same problems
  - Service location
  - Consistent exception handling
  - Parameterizing application code…
- J2EE "out of the box" does not provide a complete (or ideal) programming model
- Result: many in-house frameworks
  - Expensive to maintain and develop
  - Better to share experience across many projects


R.I.P.
In-House Framework
1998-2002

# Open Source Frameworks

- Responsible for much innovation in last 2–3 years
  - Flourishing open source is one of the great strengths of the Java platform
- Successful projects are driven by actual common problems to be solved
- Ideally placed to learn from collective developer experience
- Several products aim to simplify the development experience and remove excessive complexity from the developer's view

# How Do Lightweight Containers Work?

- Inversion of Control/Dependency Injection
  - Sophisticated configuration for POJOs
- Aspect Oriented Programming (AOP)
  - Provide declarative services to POJOs
  - Out-of-the box (transaction management, security) or custom (auditing)
- Aim to provide a consistent framework for development
- The Spring Framework and HiveMind are the most compelling offerings
  - Spring is the most complete, mature and popular

# Agenda

- "Agile" J2EE technology
  - Why we can't settle for "business as usual"
- The lightweight container movement
- **Enabling technologies**
  - Dependency Injection
  - Aspect Oriented Programming (AOP)
- The Spring Framework

# What Is Inversion of Control?

- Hollywood Principle
  - *Don't call me, I'll call you*
- Means that the framework calls your code, not the reverse
- Basic technique in framework design
- Example: Servlet container
  - Manages lifecycle of shared Servlet instances
  - Init and destroy callbacks

# What Is Dependency Injection?

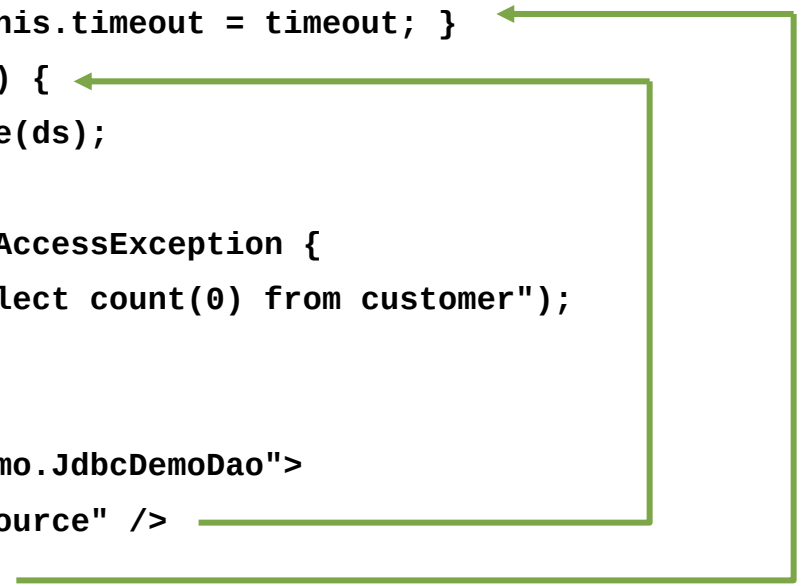
- A specialization of Inversion of Control
- The container **injects** dependencies into object instances using Java methods
- Dependencies may be collaborating objects or primitive or simple types
- This is also known as **push** configuration
  - Configuration values are **pushed** into objects, rather than **pulled** by the objects from an environment such as JNDI or a properties file

# Setter Injection

```
public class JdbcDemoDao implements DemoDao {
   private int timeout;
   private JdbcTemplate jdbcTemplate;

   public void setTimeout(int timeout) { this.timeout = timeout; }
   public void setDataSource(DataSource ds) {
       this.jdbcTemplate = new JdbcTemplate(ds);
   }
   public int countCustomers() throws DataAccessException {
       return jdbcTemplate.queryForInt("select count(0) from customer");
   }

<bean id="demoDao" class="com.mycompany.demo.JdbcDemoDao">
    <property name=“dataSource" ref=“dataSource" />
    <property name="timeout" value="30" />
</bean>
```

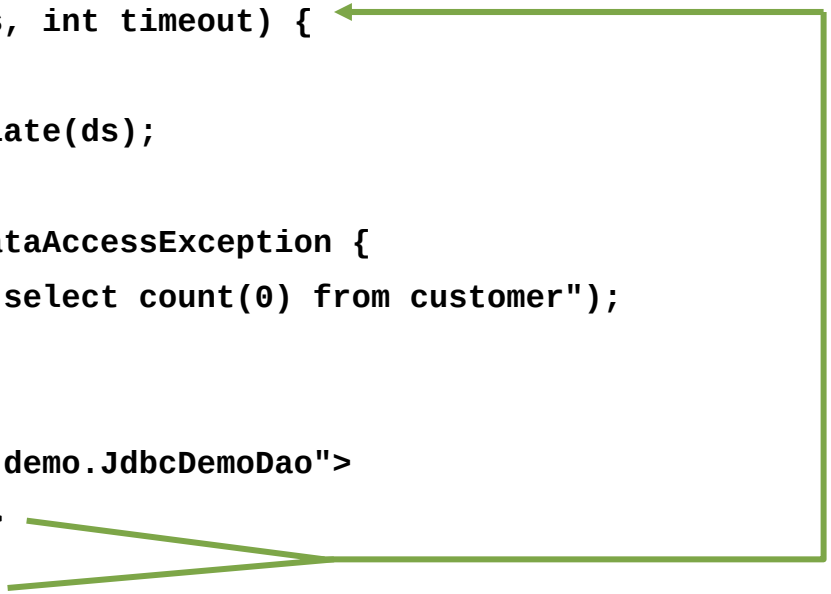


# Constructor Injection

```
public class JdbcDemoDao implements DemoDao {
   private int timeout;
   private JdbcTemplate jdbcTemplate;

   public void JdbcDemoDao(DataSource ds, int timeout) {
       this.timeout = timeout;
       this.jdbcTemplate = new JdbcTemplate(ds);
   }
   public int countCustomers() throws DataAccessException {
       return jdbcTemplate.queryForInt("select count(0) from customer");
   }

<bean id="demoDao" class="com.mycompany.demo.JdbcDemoDao">
    <constructor-arg ref=“dataSource" />
    <constructor-arg value="30" />
</bean>
```



# Why Is Dependency Injection Different?

- Configuration requires no container API
- Can use existing code that has no knowledge of the container
  - Component objects just need to expose appropriate bean property setters and/or constructor arguments
- No dependency on a specific container
  - Minimizes framework lock-in
  - Allows framework to evolve independently of application code

# Why Is Push Better Than Pull?

- No more ad-hoc lookup
- Code is self-documenting, describing its own dependencies
- Can apply consistent configuration management strategy everywhere
- Easy to unit test
  - No JNDI to stub, properties files to substitute, RDBMS data to set up
  - Simply instantiate class in a JUnit test and use setters or constructors
- Reduces the cost of programming to interfaces, rather than classes, to almost zero

# Advanced Dependency Injection

- DI is a simple but very powerful concept, but needs a sophisticated implementation to achieve its full potential
  - Instance management: shared instance, pooled, etc.
  - Ability to handling list, map, and set properties
  - Support for type conversion with property editors
  - Non-intrusive lifecycle callbacks
  - Instantiation via factory methods
- FactoryBean concept adds a level of indirection
  - Enables an object to return another type of object
  - Examples
    - JndiObjectFactoryBean
    - LocalSessionFactoryBean

# Isolation From Environment

```
<bean id="dataSource"
   class="org.apache.commons.dbcp.BasicDataSource"
   destroy-method="close">
    <property name="driverClassName" value="${jdbc.driver}"/>
    <property name="url" value="${jdbc.url}"/>
    <property name="username" value="${jdbc.username}"/>
    <property name="password" value="${jdbc.password}"/>
</bean>
```

```
<bean id="dataSource"
   class="org.springframework.jndi.JndiObjectFactoryBean">
   <property name="jndiName“ value="java:comp/env/jdbc/mydb"/>
</bean>
```

# Isolation From Environment

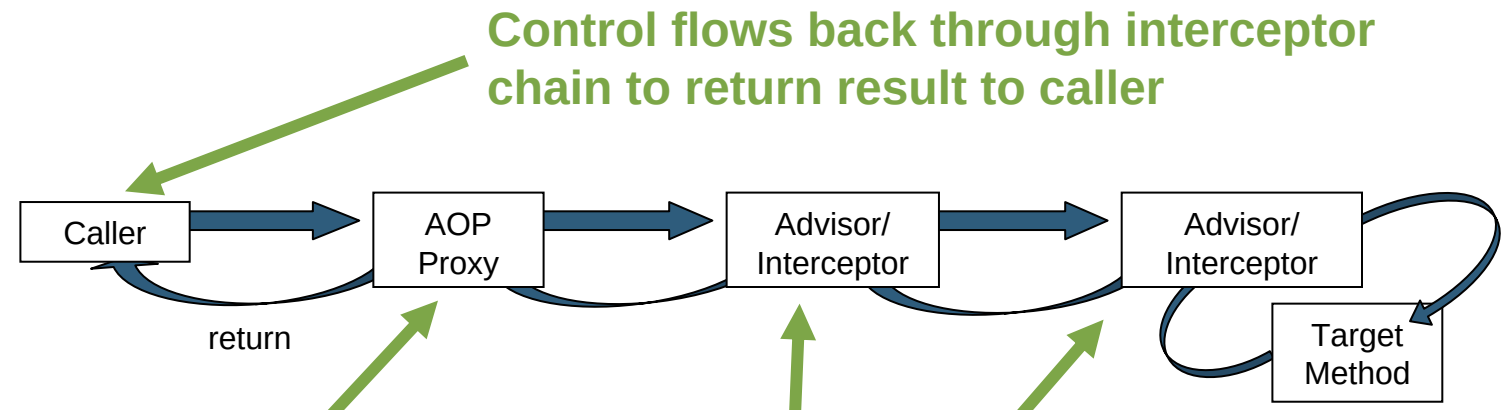
```
<bean id="dataSource"
   class="org.apache.commons.dbcp.BasicDataSource"
   destroy-method="close">
    <property name="driverClassName" value="${jdbc.driver}"/>
    <property name="url" value="${jdbc.url}"/>
    <property name="username" value="${jdbc.username}"/>
    <property name="password" value="${jdbc.password}"/>
</bean>
```

```
<bean id="dataSource"
   class="org.springframework.jndi.JndiObjectFactoryBean">
   <property name="jndiName“ value="java:comp/env/jdbc/mydb"/>
</bean>
```

# What Is AOP?

- Paradigm for modularizing crosscutting code
  - Code that would otherwise be scattered across multiple methods or objects can be gathered in one place
- In the simplest case, think about **interception**
  - Callers invoke a proxy
  - A chain of interceptors decorate the method call as execution flows toward the target
  - Interceptors provide services such as transaction management or security checks
  - These services are often specified declaratively

# AOP Interception

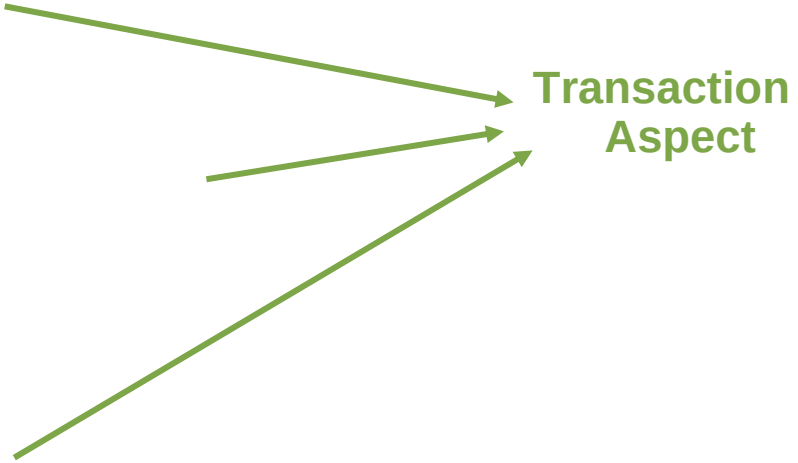


# AOP

- Ideal for addressing common enterprise concerns, such as
  - Transaction management
  - Security
- …and custom concerns such as
  - Auditing
  - Managing resources
  - Invoking a stored procedure at the beginning and end of a transaction

# A Problem Solved by AOP

```
public void sampleMethod() {
  try {
     // start transaction
     // Do real work
  } catch (SomeException ex) {
     // note transaction result,
     // even if rethrowing
  } catch (OtherException ex) {
     // …
  } finally {
    // commit or rollback
    // depending on result
  }
}
```



# AOP in J2EE: "Incremental AOP"

- Proxy-based AOP
  - Callers see a proxy, which wraps the target object
  - May use a JDK **dynamic proxy** or a byte code generation library
- Think of it as EJB++ rather than—AspectJ
- **Such use of AOP is not experimental**
  - EJB™ CMT proves the value proposition
  - Merely opening it up and generalizing it

# DI + AOP Delivers the POJO Ideal

- Dependency Injection allows us to configure objects without an invasive API
- AOP enables us to deliver enterprise and other services declaratively
- Neither of them is intrusive: can work with all kinds of target objects
- No need to sacrifice any power

# Agenda

- "Agile" J2EE technology
  - Why we can't settle for "business as usual"
- The lightweight container movement
- Enabling technologies
  - Dependency Injection
  - Aspect Oriented Programming (AOP)
- **The Spring Framework**

# The Spring Framework

- Open source project
  - Apache 2.0 license
  - 21 developers
  - Interface21 lead development effort, with seven committers (and counting), including the two project leads
- Aims
  - Simplify J2EE development
  - Provide a comprehensive solution to developing applications built on POJOs
  - Provide services for applications ranging from simple web apps up to large financial/“enterprise” applications

# The Spring Framework: Unique Capabilities

- Declarative transaction management for POJOs with or without JTA
- Unique consistent approach to data access, with common exception hierarchy
  - Greatly simplifies working with JDBC™ technology, TopLink, Hibernate, iBATIS, JDBC and other supported APIs
- IOC/AOP integration
- Powerful consistent approach to remoting across multiple protocols
- Integration with many popular third-party products
- Consistency makes Spring more than the sum of its parts

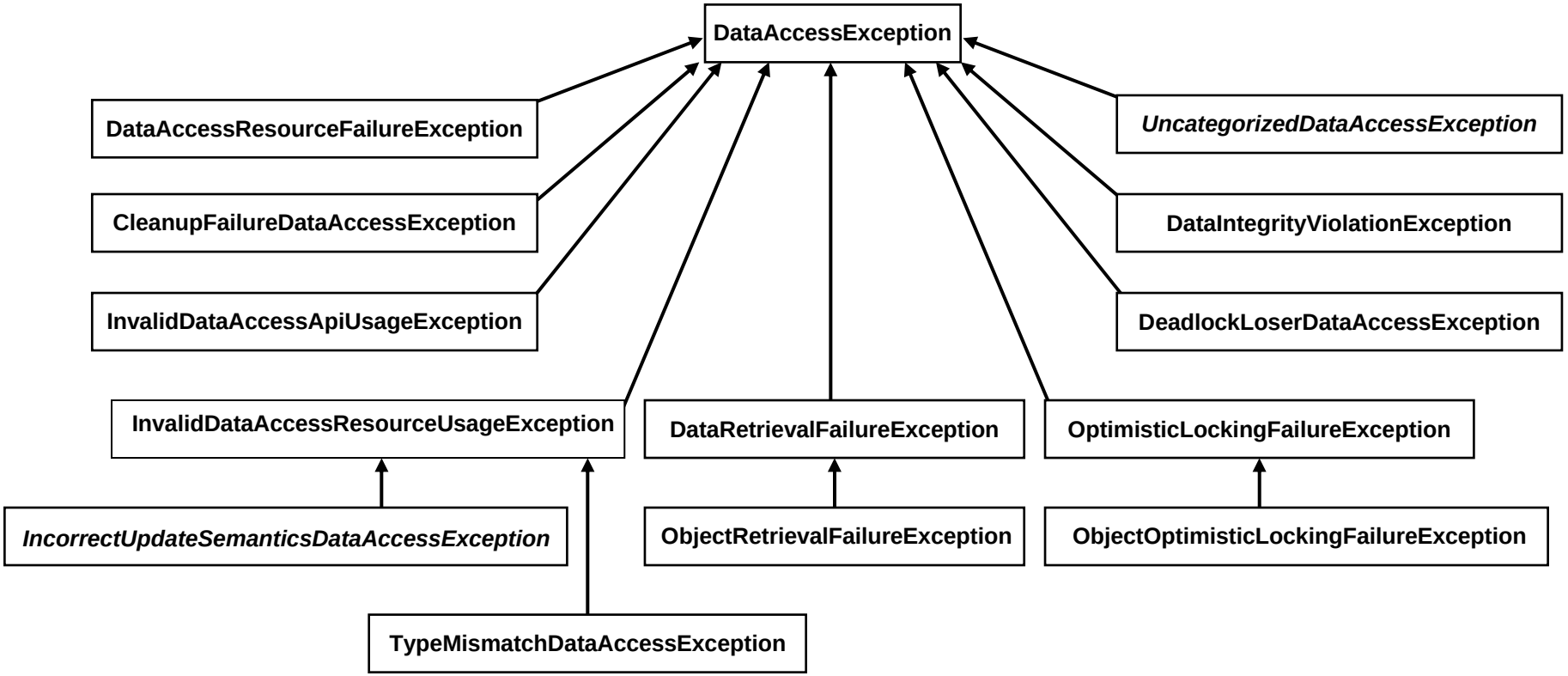
# Configuration Management

- Leading IOC container: support for all types of IOC
  - Dependency Injection: Setter, Constructor, Method
  - Dependency Lookup
- Container is very lightweight
  - No noticeable startup overhead
- Very powerful way to configure application objects
  - Down to as fine a level of granularity as desired
- Applications become very easy to unit test
  - TDD works!
- Container offers many value adds, such as
  - Instance pooling, hot swapping of instances
  - JMX-enabling of managed POJO instances

# Data Access

- Unique data access abstraction
- Consistent abstraction across all data access technologies
- Consistent, sophisticated exception handling
- Decouples DAO interface method signatures from details of persistence technology
  - No more **`InventoryDAOException`**
    - Such classes add no value; should have a generic solution
  - No more looking inside **`SQLException`**

# Data Access: Exception Hierarchy



# DAO Interfaces: A Good Practice Made Easy

```
public interface PetDao {

    Pet loadPet(int id)
        throws DataAccessException;

    void persist(Pet newPet)
        throws DataAccessException;

    int getPetCount()
        throws DataAccessException;
}
```

# Data Access: JDBC

```
public Pet loadPet(int id) throws DataAccessException {
   return (Pet) getJdbcTemplate().queryForObject(
      "SELECT owner.id, …, name FROM pets WHERE id=?",
      new Object[] { new Integer(id) },
      new RowMapper() {
         public Object mapRow(ResultSet rs, int rowNum)
               throws SQLException {
            Pet pet = new Pet();
            pet.setId(rs.getInt("id"));
            pet.setName(rs.getString("name"));
            return pet;
      }
   });
}
```

# Data Access: ORM

```
public Pet loadPet(int id) throws DataAccessException {
    return (Pet) getTopLinkTemplate().
        readAndCopy(Pet.class, new Integer(id));
}
```

- Example shows TopLink, but Spring supports all leading persistence technologies
- Don't need to worry about managing TopLink Session/UnitOfWork
- Transaction management also taken care of…

# Transaction Management

- Spring provides both **programmatic** and **declarative** transaction management
- **`PlatformTransactionManager`** is key abstraction
  - DataSourceTransactionManager
  - TopLinkTransactionManager
  - JtaTransactionManager
  - WebLogicJtaTransactionManager
  - etc.
- Works in any environment
- More powerful for programmatic use than JTA

# Declarative Transaction Management

- Most popular option
- Similar to EJB CMT, but with several important advantages
  - Can apply transaction management to any POJO
    - No need for EJB interfaces or annotations
  - Works in any environment
    - Works with JTA, but not tied to JTA
  - Supports **rollback rules**
    - Unique capability to **declaratively specific rollback on particular exceptions**
  - Supports savepoints and nested transactions for participating resources

# Declarative Transaction Management: XML Configuration

```
<bean id="petStoreTarget" class="petstore.dao.JdbcPetDao">
   ...
</bean>

<bean id="petStore" class="org...TransactionProxyFactoryBean">

   <property name="transactionManager
      ref="transactionManager“/>

   <property name="target" ref="petStoreTarget"/>

   <property name="transactionAttributes">
     <props>
       <prop key="create*">PROPAGATION_REQUIRED,
          -DuplicateOrderIdException</prop>
       <prop key="update*">PROPAGATION_REQUIRED</prop>
       <prop key="*">PROPAGATION_REQUIRED,readOnly</prop>
     </props>
   </property>

</bean>
```

# Transaction Management: Java 5 Annotations

```
@Transactional(readOnly=true)
public interface PetStoreFacade {

   @Transactional(readOnly=false,
    rollbackFor=DuplicateOrderIdException.class)
   void createOrder(Order order)
       throws DuplicateOrderIdException ;

   List loadPets(Criteria criteria);
}
```

# Other Areas

- Spring is a very extensive framework
  - Can nevertheless be used in a modular fashion
- We've only covered some of the core features, and those only very briefly
- Other important areas include
  - Spring MVC
    - Powerful MVC framework
  - Remoting
  - Scheduling
  - Spring JMX™

# Spring and Integration

- **Spring is essentially an integration platform**
  - Aims to provide a POJO model in whatever environment, with whatever services
- Minimal system requirements
  - Spring runs in Java 1.3 and above
  - Takes advantage of 1.4 and 5.0 features if available
  - Spring offers portability between different environments
- Integrates with a large number of products
  - Quartz scheduler
  - Velocity template engine
  - Jasper reports

# Spring in a J2EE Environment

- Does not violate J2EE programming restrictions
- Provides services on any J2EE application server
  - Runs well on current, stable servers such as WebSphere 4.0–6.0 and WebLogic 6.1-9.0
- Although Spring is an alternative to EJB in many cases, it provides services for invoking and implementing EJBs
  - Codeless EJB proxies, exposing a POJO interface
  - Implementing EJBs that internally use Spring

# Spring in a Web Container

- Use with a Web container such as Tomcat or Jetty
- Can provide declarative transaction management without EJB
- Java can compete in even entry level Web application architecture with PHP and other technologies
  - …but not on Blueprints style architecture
- Can scale up to a full-blown application server, without changing Java code
  - No need to make a long-term choice between **local** or **global** transactions

# Spring Beyond the Server Side

- Spring Rich
  - Simplifies Swing development by using Spring lightweight container on the client side
  - Spring eases accessing server-side services
- **`org.springframework.test`** package
  - Support for integration testing without deployment to an application server
  - Must always deploy regularly and test against your deployed environment, but the more feedback you can get during development, the better your productivity

# The Future

- Spring has established a rich and growing ecosystem
- Won't EJB 3.0 deliver the benefits of a lightweight container?
  - Perhaps some of them, but it's not there yet
  - Not a general-purpose IoC container comparable to Spring
  - Not applicable in an equally wide variety of contexts
    - IOC is much broader in applicability than EJB
  - Limited interception capability not close enough to true AOP
  - EJB 3.0 borrows some features from lightweight containers
    - Travelling in the same direction

# Who's Using Spring?

- Spring is widely used in many industries, including…
  - Banking
    - Transactional Web applications, message-driven middleware
    - Retail and investment banking
  - Scientific research
  - Defence
  - A growing number of Fortune 500 companies
  - High volume Web sites
  - **Significant enterprise usage, not merely adventurous early adopters**

# Summary

- J2EE development can and should be simpler
  - Priorities include testability and simplified API
  - Should move to a POJO model
  - Lightweight containers make this reality today!
- Two key enabling technologies
  - Dependency Injection
  - AOP
- Spring is the leading lightweight container
  - Robust and mature
  - Makes J2EE development much simpler
  - Does not mean sacrificing the power of the J2EE platform

# For More Information

- Sessions
  - Spring and JSF Technology: Synergy or Superfluous?
- Links
  - www.springframework.org
  - http://forum.springframework.org
  - http://www.springframework.com
- Books
  - Professional Spring Development (Wrox)
  - Pro Spring (Apress)
  - J2EE Development without EJB (Wrox)
  - …and many others

# Q&A

Rod Johnson

Juergen Hoeller

# Submit Session Evaluations for Prizes!

## Your opinions are important to Sun

- You can win a $75.00 gift certificate to the on-site Retail Store by telling Sun what you think!
- Turn in completed forms to enter the daily drawing
- Each evaluation must be turned in the same day as the session presentation
- Five winners will be chosen each day (Sun will send the winners e-mail)
- Drop-off locations: give to the room monitors or use any of the three drop-off stations in the North and South Halls

Note: Winners on Thursday, 6/30, will receive and can redeem certificates via e-mail

INTERFACE21
Spring from the source

# The Spring Framework: Introduction to Lightweight J2EE™ Architecture

**Rod Johnson / Juergen Hoeller**

CEO / CTO

Interface21

www.springframework.org

Session TS-7695